

Update

Donghun Jung

19 Feb 2026

Department of Physics, Sungkyunkwan University

QuiME Lab, Center for Quantum Technology, Korea

Institute of Science Technology



Outline

1. DDrf

- Work Flow
- Numerical Simulation based on Taminiau Paper
- Numerical Simulation: Side-peak Problem

2. Post-Selection

- Architecture / File Structure
- Density Matrix Analysis

3. Distributed Quantum Machine Learning

- Architecture / File Structure
- Dataset: 9-dimensional clustered binary classification
- Data Encoding: Ising-type feature map
- Potential References

Work Flow

1. Comparative Study between CPMG and DDrf (Theory/Numeric)
 - **[Pending] Poster work** ← (Hun)
2. Enhanced (Hybrid) DDrf gate (Theory/Numeric)
 - [Completed] Tried many ideas... ← (J. J)
 - [Ongoing] Alternating Ω_{RF} for odd- and even-numbered pulses ← (J. J)
3. DDrf Spectroscopy (Experiment)
 - **[Completed] Numerical Simulation based on Taminiau Paper** ← (Hun)
 - [Ongoing] Experiment ← (Dr. Lee)
 - **[Ongoing] Numerical Simulation; side-peak problem** ← (Hun)
4. Multi-qubit Control (Numeric/Experiment)
 - [Pending]

Numerical Simulation based on Taminiau Paper

As the nuclear spin quantization axis is dependent on the electron spin state, the Hamiltonian becomes

$$\begin{aligned} H &= |0\rangle\langle 0| \otimes H_0 + |1\rangle\langle 1| \otimes H_1 \\ H_0 &= \omega_0 I_z + 2\Omega_{\text{RF}} \cos(\omega_{\text{RF}}t + \phi) I_x \\ H_1 &= \omega_1 \tilde{I}_z + 2\Omega_{\text{RF}} \cos \beta \cos(\omega_{\text{RF}}t + \phi) \tilde{I}_x + 2\Omega_{\text{RF}} \sin \beta \cos(\omega_{\text{RF}}t + \phi) \tilde{I}_z \end{aligned}$$

where

$$\begin{aligned} \cos \beta &= \frac{\omega_0 - A_{\perp}}{\omega_1} & \tilde{I}_z &= \cos \beta I_z + \sin \beta I_x \\ \sin \beta &= \frac{A_{\perp}}{\omega_1} & \tilde{I}_x &= \cos \beta I_x - \sin \beta I_z. \end{aligned}$$

Numerical Simulation based on Taminiau Paper

Two rotating frames are used with respect to the electron spin state:

$$R_0(t) = e^{i\omega_{\text{RF}}tI_z} \quad R_1(t) = e^{i\omega_{\text{RF}}t\tilde{I}_z}$$

In the rotating frame, each Hamiltonian becomes:

$$\begin{aligned} H_0 \rightarrow H'_0 &= R_0(t)(H_0 - \omega_{\text{RF}}I_z)R_0(t)^\dagger \\ &= (\omega_0 - \omega_{\text{RF}})I_z + \Omega_{\text{RF}}(\cos \phi I_x + \sin \phi I_y) \end{aligned}$$

$$\begin{aligned} H_1 \rightarrow H'_1 &= R_1(t)(H_1 - \omega_{\text{RF}}I_z)R_1(t)^\dagger \\ &= (\omega_1 - \omega_{\text{RF}})\tilde{I}_z + \Omega_{\text{RF}} \cos \beta (\cos \phi \tilde{I}_x + \sin \phi \tilde{I}_y) \\ &= (\omega_1 - \omega_{\text{RF}})(\cos \beta I_z + \sin \beta I_x) + \Omega_{\text{RF}} \cos \beta (\cos \beta \cos \phi I_x + \sin \phi I_y - \sin \beta \cos \phi I_z) \end{aligned}$$

In each rotating frame, the time evolution can be readily calculated as $e^{-iH_{0(1)}t}$.

Numerical Simulation based on Taminiau Paper

Then, over the full time evolution, the unitary operation can be calculated as:

$$\begin{aligned} U &= |0\rangle\langle 0| \otimes U_0 + |1\rangle\langle 1| \otimes U_1 \\ U_0 &= R_0(4N\tau)^\dagger e^{-iH'_0\tau} R_0((2N-1)\tau) R_1((2N-1)\tau)^\dagger e^{-iH'_1 2\tau} R_1((2N-3)\tau) R_0((2N-3)\tau)^\dagger e^{-iH'_0\tau} \dots \\ &\quad \dots e^{-iH'_0\tau} R_0(3\tau) R_1(3\tau)^\dagger e^{-iH'_1 2\tau} R_1(\tau) R_0(\tau)^\dagger e^{-iH'_0\tau} R_0(0) \\ U_1 &= R_1(4N\tau)^\dagger e^{-iH'_1\tau} R_1((2N-1)\tau) R_0((2N-1)\tau)^\dagger e^{-iH'_0 2\tau} R_0((2N-3)\tau) R_1((2N-3)\tau)^\dagger e^{-iH'_1\tau} \dots \\ &\quad \dots e^{-iH'_1\tau} R_1(3\tau) R_0(3\tau)^\dagger e^{-iH'_0 2\tau} R_0(\tau) R_1(\tau)^\dagger e^{-iH'_1\tau} R_1(0) \end{aligned}$$

It is worth mentioning that

- $\Omega_{\text{RF}} \rightarrow 0, \omega_{\text{RF}} = \omega_1$: CPMG at a certain $\tau \simeq \frac{(2k-1)\pi}{2\omega_0 + A_{\parallel}}$
- $\beta \rightarrow 0, \omega_{\text{RF}} = \omega_1 > \omega_0$: DDrf(2019)
- $\Omega_{\text{RF}} \rightarrow \frac{\Omega_{\text{RF}}}{\cos \beta}$: Jiwon's idea
- $\tau = \frac{(2k-1)\pi}{2\omega_0 + A_{\parallel}}$: Hybrid DDrf

Numerical Simulation based on Taminiau Paper

The paper [Phys. Rev. X **9**, 031045 (2019)] showed that the DDrf gate offers the additional benefit of detecting spins with small $A_{\perp}$.

Procedure:

1. $\frac{\pi}{2}$ -pulse rotates electron spin to $|+\rangle$.
2. DDrf Gate with fixed N and τ , resulting in $U = |0\rangle\langle 0| \otimes U_0 + |1\rangle\langle 1| \otimes U_1$
3. $\frac{\pi}{2}$ -pulse is applied to electron spin with varying phase ϕ .
 - In our experiment, we measure P_x , the projection onto $|+\rangle$ ($\phi = \frac{\pi}{2}$).

Numerical Simulation based on Taminiau Paper

Results:

1. the expectation value becomes $P_x = \frac{1}{2} + \frac{1}{4}\Re(\text{Tr}U_0U_1^\dagger)$
2. Extended to an N -qubit simulation, $P_x = \frac{1}{2} + \frac{1}{2^{N+1}}\Re(\text{Tr}U_0U_1^\dagger)$ where

$$\text{Tr}U_0U_1^\dagger = \prod_{i=1}^N \text{Tr}U_0^iU_1^{i\dagger}.$$

3. In the Taminiau paper, the amplitude is $A = \frac{1}{2^{N+1}}|\text{Tr}U_0U_1^\dagger|$.
4. At the resonant frequency ($\omega_{\text{RF}} = \omega_1$), peaks are observed.
5. Peaks also appear at off-resonant conditions ($\omega_{\text{RF}} = \omega_1 + \frac{2\pi m}{\tau}$, $m \in \mathbb{Z}$) due to the same phase accumulation.

Numerical Simulation based on Taminiau Paper



Numerical Simulation based on Taminiau Paper



Numerical Simulation based on Taminiau Paper



Numerical Simulation based on Taminiau Paper

Problems

1. **[Resolved]** Peaks were not observed for all resonant frequencies (arbitrary m).

Note: The strengths of the off-resonant interactions are set by the Rabi frequency in combination with the detuning of the RF field ω_{RF} from both ω_0 and ω_1 .

2. **[Resolved]** The peaks corresponding to ω_{+1} and ω_{-1} must be different.

Already discussed in a previous meeting (Dr. Lee)

3. **[Ongoing]** Side-peak problem.

Numerical Simulation; side-peak problem

In the Taminiau paper, given τ , Ω_{RF} , N , and $\omega_{\text{RF}} = \omega_1$, the DDrf gate operation is

$$U = |0\rangle\langle 0| \otimes U_0 + |1\rangle\langle 1| \otimes U_1$$

where:

$$\begin{aligned} U_0 &= R_z(N(\omega_L - \omega_1)\tau) \cdot R_\phi(N\Omega_{\text{RF}}\tau) \\ U_1 &= R_z(N(\omega_L - \omega_1)\tau) \cdot R_\phi(-N\Omega_{\text{RF}}\tau) \end{aligned}$$

where $R_z(\theta) = e^{-i\theta I_z}$ and $R_\phi(\theta) = e^{-i\theta(\cos \phi I_x + \sin \phi I_y)}$.

[Observation] The operation becomes unconditional when $N\Omega_{\text{RF}}\tau = 2\pi$, suggesting a flat spectroscopy signal.

Numerical Simulation; side-peak problem



Numerical Simulation; side-peak problem



Numerical Simulation; side-peak problem



Numerical Simulation; side-peak problem

[**Analysis**] DDrf spectroscopy can be understood from the perspective of Rabi oscillations. Although an analytical form of Ω_{eff} is difficult to derive (though $\Omega_{\text{eff}} \propto \Omega_{\text{RF}}$), when the RF frequency is detuned from resonance by $\delta = \omega_1 - \omega_{\text{RF}}$, the generalized Rabi frequency is

$$\Omega_{\text{gen}} = \sqrt{\delta^2 + \Omega_{\text{eff}}^2}$$

The signal might take form:

$$P_x \simeq 1 - \underbrace{\frac{\Omega_{\text{eff}}}{\Omega_{\text{gen}}}}_{\text{Lorentzian envelope}} \underbrace{\sin^2 \frac{\Omega_{\text{gen}} 2N\tau}{2}}_{\text{oscillation}}$$

According to this analysis, the bandwidth scales as $\sim \frac{1}{2N\tau}$.

Numerical Simulation; side-peak problem

[**Suggestion**] Apply pulse shaping (e.g., a Gaussian envelope).

→ Ω_{RF} becomes time-dependent.

The time-evolution method used here *assumes* Ω_{RF} is time-independent, as required by the RWA.

$$\begin{aligned} U &= |0\rangle\langle 0| \otimes U_0 + |1\rangle\langle 1| \otimes U_1 \\ U_0 &= R_0(4N\tau)^\dagger e^{-iH'_0\tau} R_0((2N-1)\tau) R_1((2N-1)\tau)^\dagger e^{-iH'_1 2\tau} R_1((2N-3)\tau) R_0((2N-3)\tau)^\dagger e^{-iH'_0\tau} \dots \\ &\quad \dots e^{-iH'_0\tau} R_0(3\tau) R_1(3\tau)^\dagger e^{-iH'_1 2\tau} R_1(\tau) R_0(\tau)^\dagger e^{-iH'_0\tau} R_0(0) \\ U_1 &= R_1(4N\tau)^\dagger e^{-iH'_1\tau} R_1((2N-1)\tau) R_0((2N-1)\tau)^\dagger e^{-iH'_0 2\tau} R_0((2N-3)\tau) R_1((2N-3)\tau)^\dagger e^{-iH'_1\tau} \dots \\ &\quad \dots e^{-iH'_1\tau} R_1(3\tau) R_0(3\tau)^\dagger e^{-iH'_0 2\tau} R_0(\tau) R_1(\tau)^\dagger e^{-iH'_1\tau} R_1(0) \end{aligned}$$

Without this scheme, one would need to solve the Schrödinger equation directly, which takes tens of minutes per frequency point. For spectroscopy, I typically sweep hundreds to thousands of ω_{RF} values.

Post-selection: Architecture / File Structure

```
CFIsimulation/
├── main.py           # Entry point: parse config, build circuit, run optimization
├── config.yaml       # User-facing configuration file (all tunable parameters)
├── pyproject.toml    # Project metadata and dependencies
├── requirements.txt  # Pip dependency list
├── core/
│   ├── circuit.py   # Circuit class: assembles layers into a PennyLane QNode
│   ├── layers.py    # Layer:
│   │               # Initialization, Entangler, RamseyZ, PostSelection
│   ├── trainer.py   # Trainer class:
│   │               # CFI cost function, optimization, QFI benchmark
│   └── utils/
│       ├── arguments.py # Dataclass for typed configuration
│       │               # (circuitarguments, optarguments, etc.)
│       ├── customparser.py # YAML config parser → dataclass instances
│       └── utils.py      # Hamiltonians, dephasing, density matrix plotting,
│                           # file I/O helpers
├── dmplots/         # Output directory for density matrix visualizations
└── paramplots/      # Output directory for optimized parameter plots
```

Post-selection: Density Matrix Analysis

[ToDo] Entanglement investigation (two-qubit system) $\leftarrow$ Isn't it already done?

The density matrix is optimized via the following procedure.

1. Initialization
2. Entangler
3. Ramsey under dephasing noise
4. (Optional) Post-selection

After optimization, we obtain the optimal sensing time t_s .

The Entangler parameters are optimized to maximize the classical Fisher information (CFI). The quantum Fisher information (QFI) serves as the upper bound for the CFI.

Post-selection: Density Matrix Analysis

The circuit structure looks like the following:



Post-selection: Density Matrix Analysis

Presumably, this scheme is based on [*Preparation of metrological states in dipolar-interacting spin systems*, npj Quantum Inf **8**, 150 (2022)].



Distributed Quantum Machine Learning: Architecture / File Structure

```
DQML/
├── main.py                # Training entry point
├── autoscript.py          # Automated hyperparameter sweep runner
├── drawcircuit.py        # Circuit diagram generator
├── config.yaml            # Primary experiment configuration
├── dummy.yaml            # Config template used by autoscript.py
├── core/                  # Core quantum circuit implementation
│   ├── qcnn.py           # Top-level model + scheme builder
│   ├── blocks.py         # Mid-level: groups of layers into blocks
│   ├── layers.py         # Low-level: individual quantum operations
│   └── utils/
│       ├── arguments.py  # Dataclass definitions for config
│       ├── customparser.py # YAML config file parser
│       └── utils.py       # Training helpers, plotting, I/O
├── dataset/
│   ├── Dataset4          # Pre-generated dataset (pickle, ~655 KB)
│   ├── ClusterDataset.ipynb # Notebook that generates the dataset
│   └── test.py            # Quick dataset loading sanity check
├── pyproject.toml        # Project metadata and dependencies
├── requirements.txt      # Pip-compatible dependency list
└── uv.lock               # Locked dependency versions
```

Distributed Quantum Machine Learning: Data Set

The dataset is a **synthetic 9-dimensional clustered binary classification** problem:

1. **Define the cluster centers:** From all $2^9 = 512$ vertices of the hypercube $\{-\pi/4, +\pi/4\}^9$, randomly select 128 cluster centers. Half are assigned label 0, half label 1.
2. **Generate data points:** For each cluster, generate 64 data points ($8192 / 128 = 64$) by:
 - Sampling a point uniformly from a 9-dimensional ball of radius $\pi/4$
 - Shifting it to the cluster center (subtracting the vertex coordinates)
3. **Result:** 8192 data points, each a 9-dimensional vector with values roughly in $[-\pi/2, \pi/2]$, with binary labels.

Distributed Quantum Machine Learning: Data Set



Distributed Quantum Machine Learning: Data Set



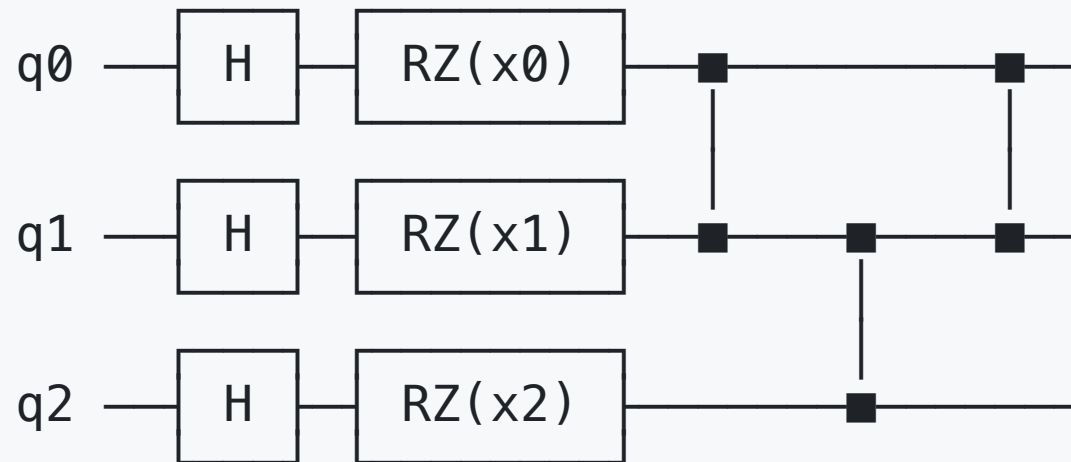
Distributed Quantum Machine Learning: Data Set



Distributed Quantum Machine Learning: Data Encoding

The embedding uses an **Ising-type feature map**, which encodes data into both single-qubit rotations and two-qubit entangling interactions:

For 3 qubits (one processor), depth=1:



$$\text{IsingZZ angle} = 0.5 * (\pi - x_i) * (\pi - x_{i+1})$$

Distributed Quantum Machine Learning: Potential References



Thank You!

Questions & Discussion